Aarya Arban

T11 – 05

Assignment No. 3

AIM – Implement the concepts of Access Modifiers, Union, Tuples using Typescript

LO2 - Understand the basic concepts of Typescript for designing web applications

THEORY:

1. Access Modifiers

Access Modifiers in TypeScript define the visibility of class members

(variables or methods). There are three main types: Modifier Description

public Accessible from anywhere (default). private Accessible only

inside the class.

protected Accessible inside the class and its subclasses.

2. Union Types

A Union allows a variable to hold more than one type. Useful when you want to accept multiple types for flexibility.

3. Tuples

A Tuple is a fixed-length array with elements of specific types. It allows storing values of different types together in a specific order.

PROGRAM:

```
// AccessModifiers.ts class
Person {     private name:
string;     protected age:
number;      public city: string;     constructor(name:
string, age: number, city: string) {        this.name = name;
this.age = age;        this.city = city;
   }
   public getDetails(): string {        return `Name: ${this.name}, Age:
${this.age}, City: ${this.city}`;
   }
}  class Employee extends Person {     private employeeId: number;
constructor(name: string, age: number, city: string, employeeId:
number) {        super(name, age, city);        this.employeeId =
employeeId;
   }
   public getEmployeeDetails(): string {        return `${this.getDetails()},
Employee ID: ${this.employeeId}`;
   }
}
// Testing access modifiers and error handling const emp = new
Employee("John Doe", 30, "New York", 12345);
try {
```

// Trying to access private member console.log(emp.name); // Error

} catch (error) { console.log("Error accessing private member 'name':", error.message);

} try

{

// Trying to access protected member console.log(emp.age); // Error

} catch (error) { console.log("Error accessing protected member 'age':", error.message);

}

console.log(emp.getEmployeeDetails()); // Accessing public method OUTPUT

\\

```
// UnionAndTuples.ts
// Union Type Example with multiple types function printIdentifier(id:
number | string | boolean): void {     if (typeof id === 'number') {
console.log(`ID (number): ${id}`);     } else if (typeof id === 'string') {
console.log(`ID (string): ${id}`);
   } else {         console.log(`ID
(boolean): ${id}`);
   }
}

// Tuple Example with multiple data types let user:
[number, string, boolean, string?]; user = [1, "Alice",
true, "Developer"]; console.log("Tuple Example:",
user);
// Testing the union type function with different
types   printIdentifier(101);   printIdentifier("XYZ-
123"); printIdentifier(true);
```

OUTPUT:


OneCompiler
PRICING
HelloWorld.ts
43dj4xugr
// UnionAndTuples.ts
// Union Type Example with multiple types
function printIdentifier(id: number | string |
if (typeof id === 'number') {
console.log(`ID (number): ${id}`);
} else if (typeof id === 'string') {
console.log(`ID (string): ${id}`);
} else {
console.log(`ID (boolean): ${id}`);
}
}
// Tuple Example with multiple data types
let user: [number, string, boolean, string?];
user = [1, "Alice", true, "Developer"];
console.log("Tuple Example:", user);
// Testing the union type function with differ
printIdentifier(101);
printIdentifier("XYZ-123");
printIdentifier(true);
STDIN
Input for the program (Optional)
Output:
Tuple Example: [1, 'Alice', true, 'Developer']
ID (number): 101
ID (string): XYZ-123
ID (boolean): true